

差分约束系统、SPFA最短路、SCC缩点

差分约束系统

差分约束系统 (System of Difference Constraints) 是一种特殊的线性规划问题，主要处理一系列变量间的**差值约束**。它通过将约束条件转化为图论中的边关系，借助最短路或最长路径算法求解，是解决带约束优化问题的重要工具。

定义

差分约束系统由 n 个变量 $x_1, x_2, \dots, x_n$ 和 m 个约束条件组成，每个约束条件形如：

$$x_j - x_i \leq c_k \quad (1 \leq i, j \leq n, 1 \leq k \leq m)$$

其中 c_k 是常数，目标是找到一组变量值满足所有约束，或判断系统无解。

解的性质

1. 解的平移不变性

若 $(x_1, x_2, \dots, x_n)$ 是一组解，则对任意常数 d ， $(x_1 + d, x_2 + d, \dots, x_n + d)$ 也是解，即：变量同时增减 d 后，差值 $x_j - x_i$ 不变，仍满足约束 $x_j - x_i \leq c_k$ 。

2. 解的存在性

系统可能无解（存在矛盾约束，如 $x_1 - x_2 \leq 1$ 且 $x_2 - x_1 \leq -2$ ）。

3. 解的极值性

若系统有解，通常要求**最大解**（每个变量尽可能大）或**最小解**（每个变量尽可能小），具体取决于问题目标。

图的转化

($\leq$ 约束) 与最短路——求解最大解

目标：在满足所有约束的前提下，使每个变量 x_i 尽可能大。

- 对约束 $x_j - x_i \leq c$ ，变形为 $x_j \leq x_i + c$ 。
- 这与图中“从节点 i 到节点 j 的最短路径松弛条件”： $dist[j] \leq dist[i] + weight(i, j)$ 一致。

转化规则为：

对约束 $x_j - x_i \leq c$, 添加一条从 i 到 j 的有向边, 权重为 c 。

($\geq$ 约束) 与最长路——求解最小解

目标: 在满足所有约束的前提下, 使每个变量 x_i 尽可能小。

- 对约束 $x_j - x_i \geq c$, 变形为 $x_j \geq x_i + c$ 。
- 这与图中“从节点 i 到节点 j 的最长路径松弛条件”: $dist[j] \geq dist[i] + weight(i, j)$ 一致。

转化规则为:

对约束 $x_j - x_i \geq c$, 添加一条从 i 到 j 的有向边, 权重为 c 。

等式约束 (=) 的转化

对约束 $x_j = x_i$, 等价于两个不等式:

$$x_j - x_i \leq 0 \quad \text{and} \quad x_i - x_j \leq 0$$

即: 添加从 i 到 j 的边 (权重 0), 和从 j 到 i 的边 (权重 0)。

无解情况的检测

系统无解的核心标志是图中存在环且环的总权重导致矛盾:

1. 最短路场景 ($\leq$ 约束)

若图中存在**负环** (环的总权重 < 0), 则约束矛盾。例如:

- 环 $i \rightarrow j \rightarrow k \rightarrow i$ 的总权重为 $c_1 + c_2 + c_3 < 0$;
- 对应约束为 $x_i \leq x_j + c_1$, $x_j \leq x_k + c_2$, $x_k \leq x_i + c_3$;
- 合并得 $x_i \leq x_i + (c_1 + c_2 + c_3)$, 即 $0 \leq \text{负数}$, 矛盾。

2. 最长路场景 ($\geq$ 约束)

若图中存在**正环** (环的总权重 > 0), 则约束矛盾。例如:

- 环 $i \rightarrow j \rightarrow i$ 的总权重为 $c_1 + c_2 > 0$;
- 对应约束 $x_j \geq x_i + c_1$, $x_i \geq x_j + c_2$;
- 合并得 $x_i \geq x_i + (c_1 + c_2)$, 即 $0 \geq \text{正数}$, 矛盾。

超级源点

为了确保图的连通性 (避免某些节点无法到达), 通常添加一个超级源点 (如编号 0):

- 从超级源点到所有其他节点添加一条权重为 0 的边
- 超级源点的距离初始化为 0

SPFA算法（解决负权图的算法）

作用

- 求解单源最短路径
- 检测图中是否存在负权回路（负环） 在差分约束系统中，若转化后的图存在负环，则系统无解；否则，最短路径数组 $dist[]$ 即为一组可行解。

负环检测

- 记录每个节点的入队次数 cnt
- 当某个节点的入队次数 $cnt[v] \geq n$ (n 为总节点数) 时，说明存在负环
- 特别的，当含有超级源点的时候，应该是 $cnt[v] \geq n + 1$

算法步骤

1. 初始化距离数组 $dist[]$ 为无穷大，超级源点 $dist[0] = 0$
2. 将超级源点加入队列，标记为已入队
3. 当队列非空时：
 - a. 取出队首节点 u
 - b. 遍历 u 的所有邻接边 (u, v, w)
 - c. 若 $dist[v] > dist[u] + w$ ，则更新 $dist[v]$
 - d. 更新 v 的入队次数 $cnt[v] = cnt[u] + 1$
 - e. 若 $cnt[v] \geq$ 总节点数，说明存在负环，系统无解
 - f. 若 v 未在队列中，则将其加入队列
4. 若未检测到负环，则 $dist[]$ 为可行解

例题：

[P1993 小 K 的农场](#)

思路：

原不等式	转化为标准形式	图中边
$x_a - x_b \geq c$	$x_b - x_a \leq -c$	$a \rightarrow b$, 权值 $-c$
$x_a - x_b \leq c$	$x_a - x_b \leq c$	$b \rightarrow a$, 权值 c
$x_a = x_b$	$x_a - x_b \leq 0$ 且 $x_b - x_a \leq 0$	$a \leftrightarrow b$, 权值 0

STD:

```

// #pragma GCC optimize(2)
#include <bits/stdc++.h>
#define endl "\n"
#define int long long
using namespace std;
const int N = 1e6+10;
const int inf = 1e18+10;

int n,m;
vector<pair<int,int>> g[N];
int dis[N],cnt[N],vis[N];
void solve()
{
    cin >> n >> m;
    for(int i=1;i<=m;i++)
    {
        int op;cin >> op;
        if(op==1)
        {
            int a,b,c;cin >> a >> b >> c;
            g[a].push_back({b,-c});
        }
        else if(op==2)
        {
            int a,b,c;cin >> a >> b >> c;
            g[b].push_back({a,c});
        }
        else if(op==3)
        {
            int a,b;cin >> a >> b;
            g[a].push_back({b,0});
            g[b].push_back({a,0});
        }
    }
    for(int i=1;i<=n;i++)
    {
        dis[i] = inf;
        g[0].push_back({i,0});
    }
    queue<int> q;
    dis[0]=0;
    vis[0]=true;
    q.push(0);
    while(!q.empty())
    {
        int u = q.front();
        q.pop();
        vis[u]=false;
        for(auto [v,w] : g[u])
        {

```

```

        if(dis[v] > dis[u]+w)
        {
            dis[v] = dis[u]+w;
            cnt[v] = cnt[u]+1;
            if(cnt[v]>=n+1)
            {
                cout << "No" << endl;
                return;
            }
            if(!vis[v])
            {
                q.push(v);
                vis[v]=true;
            }
        }
    }
    cout << "Yes" << endl;
}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    int _ = 1;
    //cin >> _;
    while(_--)
    {
        solve();
    }
    return 0;
}

```

总结

1. **核心转化**: 差分约束 $x_j - x_i \leq c \rightarrow$ 图中边 $i \rightarrow j$ (权值 c)
2. **可行性判断**: 通过SPFA检测负环, 存在负环则无解
3. **超级源点**: 确保图的连通性, 避免漏检负环

SCC缩点 (基于Tarjan)

为什么需要缩点?

缩点 (将强连通分量 SCC 收缩为单个节点) 是为了解决环结构对约束的影响, 具体原因如下:

1. 环结构的约束矛盾风险:

若图中存在环（如 $A \rightarrow B \rightarrow C \rightarrow A$ ），且环中存在权重为 1 的边（如 $A \rightarrow B$ 权重 1 表示 $A < B$ ， $B \rightarrow C$ 权重 1 表示 $B < C$ ， $C \rightarrow A$ 权重 1 表示 $C < A$ ），则会形成矛盾（ $A < B < C < A$ ），此时问题无解。缩点能快速检测这类矛盾（同一 SCC 内存在权重 1 的边则矛盾）。

2. 简化图结构，便于求解：

若图带环，则无法直接求最长路径（若环中存在正权重边，最长路径会无限大）。

缩点后，原图转化为有向无环图（DAG）（SCC 之间的边不会形成环），DAG 可通过拓扑排序高效求解最长路径。

3. 合并等价节点：

同一 SCC 中的节点相互可达（如 $A = B$ ， $B = C$ 则 A、B、C 在同一 SCC），它们必须满足所有相互约束，最终会被分配相同的数值来满足约束。缩点后可将整个 SCC 视为一个节点，减少计算量。

缩点的原理

强连通分量（SCC）见 [Tarjan 算法](#)

缩点过程：

- 用 Tarjan 算法找到所有 SCC 后，将每个 SCC 收缩为一个“超级节点”（编号为 `cnt`），并记录每个 SCC 包含的节点数（`size[cnt]`）。

新图构建：

对于原图中的每条边（ $u \rightarrow v$ ）（权重 w ）：

- 若 u 和 v 属于同一 SCC：忽略（内部约束已通过 SCC 处理）。
- 若 u 和 v 属于不同 SCC：在新图中添加超级节点之间的边（`id[u] → id[v]`，权重 `w`），并记录新图中节点的入度。
- 缩点后的新图一定是 DAG（无环），因此新图可安全地用拓扑排序求解。

例题：

[P3275 \[SCOI2011\] 糖果](#)

思路

操作类型	差分约束不等式	对应图边	说明
$x = 1$	$d[A] - d[B] \leq 0$ 且 $d[B] - d[A] \leq 0$ (即 $d[A] = d[B]$)	$A \rightarrow B$ (权重 0) 、 $B \rightarrow A$ (权重 0)	相等等价于 $d[A] = d[B]$
$x = 2$	$d[B] - d[A] \geq 1$	$A \rightarrow B$ (权重 1)	$A < B$ 等价于 $d[B] \geq d[A] + 1$
$x = 3$	$d[A] - d[B] \geq 0$	$B \rightarrow A$ (权重 0)	$A \geq B$ 等价于 $d[A] \geq d[B] + 0$
$x = 4$	$d[A] - d[B] \geq 1$	$B \rightarrow A$ (权重 1)	$A > B$ 等价于 $d[A] \geq d[B] + 1$
$x = 5$	$d[B] - d[A] \geq 0$	$A \rightarrow B$ (权重 0)	$A \leq B$ 等价于 $d[B] \geq d[A] + 0$

STD:


```

// #pragma GCC optimize(2)
#include <bits/stdc++.h>
#define endl "\n"
#define int long long
using namespace std;
const int N = 1e6+10;
const int inf = 1e18+10;

int n,k;
vector<pair<int,int>> og[N];
vector<pair<int,int>> g[N];

int inStack[N];
int dfn[N],low[N];
stack<int> st;
int scc[N],sz[N];
int timestramp;
int cnt;

int dis[N];
int in[N];
void tarjanDfs(int u)
{
    st.push(u);
    low[u] = dfn[u] = ++timestramp;
    inStack[u] = true;
    for(auto [v,w] : og[u])
    {
        if(!dfn[v])
        {
            tarjanDfs(v);
            low[u] = min(low[v],low[u]);
        }
        else if(inStack[v])
            low[u] = min(low[v],low[u]);
    }
    if(dfn[u]==low[u])
    {
        cnt++;
        int y;
        do
        {
            y = st.top();
            inStack[y] = false;
            scc[y] = cnt;
            sz[cnt]++;
            st.pop();
        }while(y!=u);
    }
}

```

```

void solve()
{
    cin >> n >> k;
    for(int i=1;i<=k;i++)
    {
        int op,a,b;cin >> op >> a >> b;
        switch (op)
        {
            case 1:
                og[a].push_back({b,0});
                og[b].push_back({a,0});
                break;
            case 2:
                og[a].push_back({b,1});
                break;
            case 3:
                og[b].push_back({a,0});
                break;
            case 4:
                og[b].push_back({a,1});
                break;
            case 5:
                og[a].push_back({b,0});
                break;
        }
    }

    for(int i=1;i<=n;i++)
        if(!dfn[i]) tarjanDfs(i);

    for(int i=1;i<=n;i++)
    {
        for(auto [v,w] : og[i])
        {
            int uid = scc[i],vid = scc[v];

            if(uid == vid and w == 1)
            {
                cout << -1;
                return;
            }

            if(uid!=vid)
            {
                g[uid].push_back({vid,w});
                in[vid]++;
            }
        }
    }
}

```

```

queue<int> q;

for(int i=1;i<=cnt;i++)
{
    if(!in[i])
    {
        q.push(i);
        dis[i] = 1;
    }
}

while(!q.empty())
{
    int u = q.front();
    q.pop();
    for(auto [v,w] : g[u])
    {
        in[v]--;
        dis[v] = max(dis[u]+w,dis[v]);
        if(!in[v]) q.push(v);
    }
}

int ans = 0;
for(int i=1;i<=cnt;i++) ans += dis[i]*sz[i];
cout << ans << endl;
}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0),cout.tie(0);
    int _ = 1;
    //cin >> _;
    while(_--)
    {
        solve();
    }
    return 0;
}

```